1. Objetivo

El objetivo de este laboratorio es aplicar persistencia políglota en Mexi Banco: usar para cada dato el motor de base de datos que mejor le queda, sin mover de lugar nada que sea dinero. En concreto:

- Agregar una caché en Redis para el saldo que se le muestra al cliente (GET /cuentas/{clabe}), con el patrón cache-aside, un TTL de 30 segundos y el borrado de la copia cada vez que cambia el saldo.
- Mudar la bitácora de avisos del servicio notificacion de PostgreSQL a MongoDB, con un documento por aviso cuya forma depende del canal (SMS, correo o push).
- Medir lo que se gana con la caché (hit ratio, latencia y lecturas a PostgreSQL) y provocar dos fallas a propósito: un evento duplicado y un dato viejo en la caché.

2. Contexto: verdad contra copia

Después de la S09 todo Mexi Banco vivía en PostgreSQL: cuentas, movimientos, transferencias, el outbox y también los avisos de notificación. Funcionaba, pero había dos datos a los que Postgres no les queda bien. El primero es el saldo que el cliente consulta: la app lo pide cada vez que alguien abre la pantalla, muchas más veces de las que cambia, y cada consulta es una lectura a la misma base que tiene que aguantar los cargos y los abonos. El segundo son los avisos: un SMS lleva teléfono y texto, un correo lleva dirección, asunto y cuerpo, y un push lleva el token del celular. En una tabla eso significa una columna por cada campo de cualquier canal (casi todas vacías) o una tabla por canal, y además nunca se actualizan: se escriben una vez y se leen enteros.

La pregunta importante no es «¿SQL o NoSQL?», sino qué dato es la verdad y qué dato es una copia. La verdad tiene que estar en un lugar que no la pierda ni la contradiga. Una copia se puede reconstruir, y por eso se vale guardarla en otro motor, con otra forma y hasta un poco atrasada.

Dato	Dónde vive	¿Verdad o copia?	Cómo se mantiene
Cuenta y saldo	PostgreSQL	Verdad	Transacción local con @Version
Movimientos	PostgreSQL	Verdad	Asientos con Idempotency-Key (S08)
Transferencias y outbox	PostgreSQL	Verdad	Saga y Outbox (S08 y S09)
Saldo para mostrar	Redis	Copia, hasta 30 s atrás	Se borra al cambiar el saldo y caduca sola
Avisos enviados	MongoDB	Verdad de la bitácora de avisos	Los escribe el consumidor de la S09

Durante todo el laboratorio seguí las tres reglas del día:

1. **Nada que sea dinero cambia de motor.** Saldo, movimientos y transferencias se quedan en Postgres.
2. **La caché muestra, nunca decide.** El cargo que revisa si alcanza el saldo lee Postgres, siempre.
3. **No escribir lo mismo en dos bases.** Sin una transacción común, tarde o temprano una se queda sin el dato (el problema que vimos en la S09).

3. Parte 0: levantar el banco y ver el punto de partida

3.1 Levantar el banco

Levanté todos los servicios con docker compose up -d (las imágenes ya estaban construidas desde casa) y revisé con docker compose ps que los trece contenedores estuvieran en estado healthy: los de la S09 más redis y mongo. Dejé encendidas las tres réplicas de cuenta, porque más adelante sirven para comprobar que todas comparten la misma caché.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose ps
mexi-banco-v06-cuenta-1          mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          31 seconds ago   Up 24 seconds (healthy)   8080/tcp
mexi-banco-v06-cuenta-2          mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          31 seconds ago   Up 24 seconds (healthy)   8080/tcp
mexi-banco-v06-cuenta-3          mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          31 seconds ago   Up 24 seconds (healthy)   8080/tcp
mexi-banco-v06-db-1              postgres:17-alpine             "docker-entrypoint.s…"   db              31 seconds ago   Up 30 seconds (healthy)   5432/tcp
mexi-banco-v06-discovery-1       mexi-banco-v06-discovery       "java -jar app.jar"      discovery       31 seconds ago   Up 30 seconds (healthy)   0.0.0.0:8761->8761/tcp, [::]:8761->8
761/tcp
mexi-banco-v06-gateway-1         mexi-banco-v06-gateway         "java -jar app.jar"      gateway         31 seconds ago   Up 24 seconds (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8
080/tcp
mexi-banco-v06-mongo-1           mongo:8.0                      "docker-entrypoint.s…"   mongo           31 seconds ago   Up 30 seconds (healthy)   127.0.0.1:27017->27017/tcp
mexi-banco-v06-movimiento-1      mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      31 seconds ago   Up 24 seconds (healthy)   8080/tcp
mexi-banco-v06-notificacion-1    mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    31 seconds ago   Up 23 seconds (healthy)   8080/tcp
mexi-banco-v06-rabbitmq-1        rabbitmq:4-management          "docker-entrypoint.s…"   rabbitmq        31 seconds ago   Up 30 seconds (healthy)   0.0.0.0:5672->5672/tcp, [::]:5672->5
672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
mexi-banco-v06-redis-1           redis:8-alpine                 "docker-entrypoint.s…"   redis           31 seconds ago   Up 30 seconds (healthy)   127.0.0.1:6379->6379/tcp
mexi-banco-v06-spei-1            mexi-banco-v06-spei            "java -jar app.jar"      spei            31 seconds ago   Up 24 seconds (healthy)   8080/tcp
mexi-banco-v06-transferencia-1   mexi-banco-v06-transferencia   "java -jar app.jar"      transferencia   30 seconds ago   Up 23 seconds (healthy)   8080/tcp
```

3.2 Redis y MongoDB responden

Con redis-cli ping Redis contestó PONG, y con mongosh la colección avisos de la base notificacion tenía 0 documentos. Es decir, Redis estaba vivo y en Mongo todavía no había ningún aviso, porque notificación seguía guardándolos en la bandeja de Postgres.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli ping
PONG

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
0
```

3.3 Cuentas de prueba y primera transferencia

Por el gateway creé las cuentas de Ana (…001, saldo inicial 1000), Beto (…002, saldo inicial 500) y Carla (…003, saldo inicial 300). Después hice una transferencia de 200 de Ana a Beto, que respondió 201 con estado COMPLETADA. Es la misma transferencia que reutilicé en toda la práctica, cambiando las CLABEs y el monto.





3.4 Medición del «antes»: cada consulta va a Postgres

Pedí el saldo de Ana diez veces seguidas y conté en el log de las tres réplicas de cuenta la línea «leida de Postgres». El conteo dio al menos 10: las diez consultas más las lecturas que hizo la saga durante la transferencia. En cada respuesta el campo leidaEn fue distinto, lo que prueba que las diez se leyeron de Postgres. En Redis, keyspace_hits y keyspace_misses estaban en 0, porque nadie lo usaba todavía.

Este es mi «antes»: diez consultas, diez lecturas a la base que además tiene que atender los cargos y los abonos.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ for i in $(seq 1 10); do curl -s http://localhost:8080/api/cuentas/00218000000000001; echo; done
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.097583151Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.157460158Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.205303912Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.253309540Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.299736671Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.347930888Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.394962642Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.441815374Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.488871529Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T22:58:04.536263272Z"}
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep -c "leida de Postgres"
12

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:0
keyspace_misses:0
```

4. Parte 1: la caché del saldo en Redis

4.1 La idea: cache-aside

Usé el patrón cache-aside: la aplicación pregunta primero a la caché y, solo si el dato no está, va a la base y deja una copia para la próxima vez.

- **Hit:** la clave cuentas::{clabe} está en Redis. Contesta Redis, el método ni siquiera se ejecuta y Postgres no se entera.
- **Miss:** la clave no está. Se ejecuta el método, se lee Postgres y Spring guarda la copia con SET ... PX 30000, es decir, vive 30 segundos.
- **Cambio de saldo:** con un cargo o un abono la copia no se actualiza, se borra con DEL después del commit. La siguiente consulta la vuelve a llenar con el saldo nuevo.

En Spring no escribí los GET, SET y DEL a mano: los generan dos anotaciones, @Cacheable en el método que lee y @CacheEvict en los métodos que cambian el saldo.

4.2 Mi código: ConfiguracionCache.java

Archivo: cuenta/src/main/java/mx/mexibanco/cuenta/ConfiguracionCache.java

```
@EnableCaching
```

```
@Configuration
public class ConfiguracionCache {

   public static final String CUENTAS = "cuentas";

   @Bean
   RedisCacheManagerBuilderCustomizer cacheDeCuentas(
         @Value("${mexibanco.cache.ttl:30s}") Duration ttl, ObjectMapper json) {
      RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
         .entryTtl(ttl)                    // 1. cada entrada vive 30 s
         .disableCachingNullValues()
         .serializeValuesWith(SerializationPair.fromSerializer(
            new Jackson2JsonRedisSerializer<>(json, CuentaVista.class))); // 2. JSON
      return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
         .transactionAware();              // 3. el DEL espera al commit
   }
}
```

Explicación:

- @EnableCaching enciende la abstracción de caché de Spring. Sin ella, @Cacheable y @CacheEvict se ignoran sin dar ningún error, así que es lo primero que hay que revisar si todas las consultas siguen saliendo de Postgres.
- Spring Boot ya arma por su cuenta un RedisCacheManager, porque el proyecto trae spring-boot-starter-data-redis. Con el RedisCacheManagerBuilderCustomizer no reemplazo ese manager: solo le ajusto la configuración de la caché cuentas.
- entryTtl(ttl): cada entrada caduca a los 30 segundos (el valor de mexibanco.cache.ttl en application.yml). Sin TTL una copia no caducaría nunca, y si alguien cambiara el saldo por fuera de la aplicación el dato viejo se quedaría para siempre. El TTL es la red de seguridad que limita cuánto puede mentir la caché.
- disableCachingNullValues(): no se guardan valores nulos en Redis.
- Jackson2JsonRedisSerializer con CuentaVista.class: el valor se guarda como JSON legible con redis-cli GET, en lugar de la serialización binaria de Java (que se ve como \xac\xed...). Uso el ObjectMapper que inyecta Spring porque ya sabe escribir fechas como leidaEn.
- transactionAware(): el DEL que produce @CacheEvict espera a que la transacción haga commit. Si borrara antes, otra consulta podría leer el saldo viejo de Postgres en esa ventana y volver a dejarlo en la caché.

4.3 Mi código: ConsultaDeSaldo.java

Archivo: cuenta/src/main/java/mx/mexibanco/cuenta/ConsultaDeSaldo.java

```
@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
public CuentaVista consultar(String clabe) {
   log.info("[consulta] {} leida de Postgres", clabe);   // en un hit ya no sale
   return cuentas.findByClabe(clabe)
      .map(CuentaVista::de)
```

```
        .orElseThrow(() -> new NoEncontradoException("No existe la CLABE " + clabe));
}
```

@Cacheable hace que, antes de ejecutar consultar, Spring busque en Redis la clave cuentas::<clabe>; el key = "#clabe" toma el parámetro del método para armar la clave, por ejemplo cuentas::0021800000000000001. En un hit regresa la copia y el método no corre, por eso la línea «leida de Postgres» ya no aparece en el log. En un miss corre el método, lee Postgres y Spring guarda el resultado con su TTL de 30 segundos.

4.4 Mi código: CuentaService.java

Archivo: cuenta/src/main/java/mx/mexibanco/cuenta/CuentaService.java

```
@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
@Transactional
public SaldoAplicado cargar(String clabe, BigDecimal monto, String clave) { ... }

@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
@Transactional
public SaldoAplicado abonar(String clabe, BigDecimal monto, String clave) { ... }
```

Agregué @CacheEvict con la misma caché y la misma llave en cargar y en abonar, junto al @Transactional que ya tenían. Como la caché es transactionAware, el borrado se registra mientras corre el método y se ejecuta hasta que la transacción hace commit. Además, por defecto @CacheEvict solo borra cuando el método termina bien: si un cargo se rechaza (por ejemplo, por saldo insuficiente) la copia no se toca, lo cual tiene sentido porque el saldo no cambió.

Lo que no toqué: CuentaService.consultar. Es la lectura que usa cargar para decidir si alcanza el saldo y siempre debe ir a Postgres. Si la cacheara, un cargo podría autorizarse con un saldo de hasta 30 segundos atrás. Por eso la consulta que solo muestra vive en otra clase, ConsultaDeSaldo. Hay una segunda razón: si pusiera @Cacheable en un método y lo llamara desde su misma clase, la llamada no pasaría por el proxy de Spring y la anotación se ignoraría sin avisar.

4.5 Comprobación: las tres réplicas comparten la caché

Reconstruí solo el servicio cuenta con docker compose up --build -d cuenta y pedí el saldo de Ana seis veces. Antes de correrlo predije una sola lectura a Postgres, porque la caché no vive dentro de cada réplica sino en un Redis que comparten las tres.

Así fue: la cabecera X-Instancia cambió entre respuestas (contestaron distintas réplicas), pero leidaEn fue el mismo en las seis. Solo la primera consulta leyó Postgres y las demás salieron de Redis, sin importar qué réplica contestara. El conteo de «leida de Postgres» en los últimos 30 segundos dio 1. Con redis-cli GET vi la copia guardada como JSON de CuentaVista (clabe, titular, saldo y leidaEn) y con TTL los segundos que le quedaban de vida, siempre menos de 30.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose up --build -d cuenta
#30 naming to docker.io/library/mexi-banco-v06-cuenta:latest done
#30 unpacking to docker.io/library/mexi-banco-v06-cuenta:latest
#30 unpacking to docker.io/library/mexi-banco-v06-cuenta:latest 0.3s done
#30 DONE 1.8s

#31 [cuenta] resolving provenance for metadata file
#31 DONE 0.0s
[+] up 8/8
 ✓ Image mexi-banco-v06-cuenta          Built
 ✓ Image mexi-banco-v06-discovery       Built
 ✓ Container mexi-banco-v06-db-1        Healthy
 ✓ Container mexi-banco-v06-discovery-1 Healthy
 ✓ Container mexi-banco-v06-redis-1     Healthy
 ✓ Container mexi-banco-v06-cuenta-1    Started
 ✓ Container mexi-banco-v06-cuenta-3    Started
 ✓ Container mexi-banco-v06-cuenta-2    Started
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ for i in 1 2 3 4 5 6; do curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 | grep -iE 'x-instancia|leidaEn'; done
X-Instancia: 5f0b06d606fa
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T23:09:39.690280276Z"}
X-Instancia: 1543e0cb4ed4
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T23:09:39.690280276Z"}
X-Instancia: b96353a4e650
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T23:09:39.690280276Z"}
X-Instancia: 5f0b06d606fa
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T23:09:39.690280276Z"}
X-Instancia: 1543e0cb4ed4
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T23:09:39.690280276Z"}
X-Instancia: b96353a4e650
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T23:09:39.690280276Z"}
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose logs --since 30s cuenta | grep -c "leida de Postgres"
1
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli GET cuentas::002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T23:12:05.491980030Z"}

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
17
```

4.6 Comprobación: la copia se borra al cambiar el saldo

En una segunda terminal dejé corriendo redis-cli MONITOR, que muestra en vivo cada comando que recibe Redis, y en la primera terminal transferí 10 de Ana a Beto. En el monitor aparecieron, en este orden:

1. GET cuentas::0021800000000000001: la saga revisa que exista la cuenta de Ana.
2. DEL cuentas::0021800000000000001: se borra la copia de Ana después del commit del cargo.
3. DEL cuentas::0021800000000000002: se borra la copia de Beto después del commit del abono.
4. GET cuentas::0021800000000000001: mi consulta posterior, que fue un miss.
5. SET cuentas::0021800000000000001 ... PX 30000: la copia nueva, con el saldo ya descontado.

Justo después de la transferencia, EXISTS dio 0 (la copia de Ana ya no estaba), y la nueva consulta trajo el saldo actualizado, 10 pesos menos, con un leidaEn nuevo. Entre medio también pueden aparecer un GET y un SET de Beto, que son las comprobaciones de la saga.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli MONITOR
1794006952.691067 [0 127.0.0.1:49590] "ping"
1794006955.968144 [0 172.19.0.13:33842] "GET" "cuentas::0021800000000000001"
1794006956.079429 [0 172.19.0.13:33842] "SET" "cuentas::0021800000000000001" "{\"clabe\":\"0021800000000000001\",\"titular\":\"Ana\
",\"saldo\":800.00,\"leidaEn\":\"2026-11-06T23:15:56.073203900Z\"}" "PX" "30000"
1794006956.085173 [0 172.19.0.14:33384] "GET" "cuentas::0021800000000000002"
1794006956.089080 [0 172.19.0.14:33384] "SET" "cuentas::0021800000000000002" "{\"clabe\":\"0021800000000000002\",\"titular\":\"Beto
\",\"saldo\":700.00,\"leidaEn\":\"2026-11-06T23:15:56.088499752Z\"}" "PX" "30000"
1794006956.270998 [0 172.19.0.9:48756] "DEL" "cuentas::0021800000000000001"
1794006956.351947 [0 172.19.0.13:33842] "DEL" "cuentas::0021800000000000002"
1794006957.717262 [0 127.0.0.1:38034] "ping"
1794006962.744199 [0 127.0.0.1:38036] "ping"
1794006963.503515 [0 127.0.0.1:38040] "EXISTS" "cuentas::0021800000000000001"
1794006967.767695 [0 127.0.0.1:51094] "ping"
1794006969.998901 [0 172.19.0.9:48756] "GET" "cuentas::0021800000000000001"
1794006970.022712 [0 172.19.0.9:48756] "SET" "cuentas::0021800000000000001" "{\"clabe\":\"0021800000000000001\",\"titular\":\"Ana\"
,\"saldo\":790.00,\"leidaEn\":\"2026-11-06T23:16:10.015753229Z\"}" "PX" "30000"
1794006972.795149 [0 127.0.0.1:51096] "ping"
1794006977.821914 [0 127.0.0.1:49770] "ping"
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"00218000000000
0001","claveDestino":"0021800000000000002","monto":10}'
{"id":2,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":10,"fecha":"2026-11-06T23:15:56.090874316Z
","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":790.00,"intentosDeCompensacion":0,"actualizada":"2026-11-06T23:15:56
.375765757Z","eventoId":"b5e488e4-9c65-474d-8194-b34ed77d0936"}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli EXISTS cuentas::0021800000000000001
0

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/0021800000000000001
{"clabe":"0021800000000000001","titular":"Ana","saldo":790.00,"leidaEn":"2026-11-06T23:16:10.015753229Z"}
```

5. Parte 1: medidas, ¿qué compré con la caché?

5.1 Hit ratio

Redis cuenta sus aciertos (keyspace_hits) y sus fallos (keyspace_misses); el hit ratio es hits / (hits + misses). Antes de cada escenario reinicié los contadores con CONFIG RESETSTAT y no hice ningún redis-cli GET a mano durante la medición, porque también cuenta.

Escenario A (100 consultas seguidas del saldo de Ana): prácticamente todas fueron hits. Solo la primera consulta puede fallar, porque es la que llena la copia, así que el hit ratio quedó alrededor del 99 %. Las 100 consultas tardan mucho menos de 30 segundos, por lo que la copia nunca alcanzó a caducar.

Escenario B (lo mismo, pero con una transferencia de 1 peso de Ana a Beto cada diez consultas): el hit ratio bajó a alrededor del 84 %. Baja porque cada transferencia borra las copias de Ana y de Beto, y la siguiente lectura de cada uno es un miss. Además, la saga consulta que existan las dos cuentas por el mismo GET cacheado, y la de Beto casi siempre falla porque la transferencia anterior la acaba de borrar. Más escrituras significan más invalidaciones y menos aciertos: la caché rinde más mientras más veces se lee un dato por cada vez que cambia.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null http://localhost:8080/api/cuentas/0021800000000000001; done

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:99
keyspace_misses:1
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001; if [ $((i % 10)) -eq 0 ];
then curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen"
:"002180000000000001","claveDestino":"002180000000000002","monto":1}'; fi; done

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:100
keyspace_misses:20
```

5.2 Latencia y lecturas a Postgres, con y sin caché

Medí 100 consultas con curl -w '%{time_total}' y calculé el promedio, la mediana y el p95 en milisegundos. Corrí la medición tres veces con la caché encendida y tres veces con la caché apagada. Para apagarla no toqué código: reinicié cuenta con CACHE_TIPO=none docker compose up -d cuenta, porque

application.yml ya lee esa variable en spring.cache.type. En cada caso conté también las lecturas a Postgres en el log de cuenta.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ T=$(date -u +%Y-%m-%dT%H:%M:%SZ); for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api
/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=
%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'; echo -n "lecturas a Postgres: "; docker compose logs --since "$T" cuenta | g
rep -c "leida de Postgres"
promedio=6.35 ms mediana=6.23 ms p95=6.69 ms
lecturas a Postgres: 1

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ T=$(date -u +%Y-%m-%dT%H:%M:%SZ); for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api
/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=
%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'; echo -n "lecturas a Postgres: "; docker compose logs --since "$T" cuenta | g
rep -c "leida de Postgres"
promedio=6.39 ms mediana=6.15 ms p95=7.72 ms
lecturas a Postgres: 0

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ T=$(date -u +%Y-%m-%dT%H:%M:%SZ); for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api
/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=
%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'; echo -n "lecturas a Postgres: "; docker compose logs --since "$T" cuenta | g
rep -c "leida de Postgres"
promedio=6.17 ms mediana=5.98 ms p95=7.11 ms
lecturas a Postgres: 0
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ CACHE_TIPO=none docker compose up -d cuenta
[+] up 6/6
 ✓ Container mexi-banco-v06-discovery-1 Healthy
 ✓ Container mexi-banco-v06-redis-1     Healthy
 ✓ Container mexi-banco-v06-db-1        Healthy
 ✓ Container mexi-banco-v06-cuenta-1    Started
 ✓ Container mexi-banco-v06-cuenta-2    Started
 ✓ Container mexi-banco-v06-cuenta-3    Started
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ T=$(date -u +%Y-%m-%dT%H:%M:%SZ); for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api
/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=
%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'; echo -n "lecturas a Postgres: "; docker compose logs --since "$T" cuenta | g
rep -c "leida de Postgres"
lecturas a Postgres: 100

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ T=$(date -u +%Y-%m-%dT%H:%M:%SZ); for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api
/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=
%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'; echo -n "lecturas a Postgres: "; docker compose logs --since "$T" cuenta | g
rep -c "leida de Postgres"
promedio=7.79 ms mediana=7.68 ms p95=8.80 ms
lecturas a Postgres: 100

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ T=$(date -u +%Y-%m-%dT%H:%M:%SZ); for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api
/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=
%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'; echo -n "lecturas a Postgres: "; docker compose logs --since "$T" cuenta | g
rep -c "leida de Postgres"
promedio=7.53 ms mediana=7.42 ms p95=8.33 ms
lecturas a Postgres: 100
```

Resultado. Con la caché encendida, de cada 100 consultas a lo mucho una llegó a Postgres; con la caché apagada llegaron las 100. En latencia la diferencia fue mucho menor: con caché la consulta salió apenas unos milisegundos más rápida en promedio y mediana. Esto pasa porque la mayor parte del tiempo de cada petición se va en el gateway, la red interna de Docker y HTTP, y una lectura de una sola fila por llave en Postgres ya es rápida. Lo que realmente compré con la caché fue **capacidad**: le quité a Postgres casi el 100 % de las lecturas de saldo, y esa es la base que tiene que atender los cargos y los abonos.

6. Parte 2: los avisos en MongoDB

6.1 Un documento por aviso, con la forma de su canal

Cada transferencia genera dos avisos, uno para quien envía y otro para quien recibe. Cada aviso es un documento con campos comunes a todos (eventoId, transferenciaId, clabe, tipo, monto, saldo, fecha y canal) más los campos de su canal. En una tabla relacional esto obligaría a tener una columna por cada campo de cualquier canal o una tabla por canal; en MongoDB cada documento tiene la forma que necesita y nadie hace ALTER TABLE. El esquema lo aplica quien lee.

Dos decisiones del contrato:

- **El _id lo pone el banco, no Mongo:** eventoId + ":" + clabe. El mismo evento con la misma CLABE siempre da el mismo _id, y Mongo no deja guardar dos documentos con el mismo _id; así un evento repetido no duplica el aviso. No basta con el eventoId porque un evento genera dos avisos (origen y destino).
- **El dinero va en Decimal128, nunca en double:** en punto flotante 0.1 + 0.2 no da exactamente 0.3, y con dinero no se vale redondear.

Pieza	Valor
Base y colección	Base notificacion, colección avisos (constante COLECCION)
_id	eventoId + ":" + clabe
Campos de todos	eventoId, transferenciaId, clabe, tipo, monto y saldo (Decimal128), fecha (la del evento), canal
Canal SMS	telefono, texto
Canal CORREO	correo, asunto, cuerpo
Canal PUSH	token, titulo, cuerpo
Índice	clabe_fecha: { clabe: 1, fecha: -1 }
Endpoint	GET /api/notificaciones/{clabe}: los últimos 10, del más nuevo al más viejo

6.2 Mi código: documento(...) en BitacoraDeAvisos.java

Archivo: notificacion/src/main/java/mx/mexibanco/notificacion/avisos/BitacoraDeAvisos.java

```
public static Document documento(EventoTransferencia evento, Destinatario destinatario,
                                 Contacto contacto) {
```

```
    Document aviso = new Document("_id", evento.eventoId() + ":" + destinatario.clabe())
        .append("eventoId", evento.eventoId().toString())
        .append("transferenciaId", evento.transferenciaId())
        .append("clabe", destinatario.clabe())
        .append("tipo", destinatario.tipo())
        .append("monto", new Decimal128(evento.monto()))      // dinero: Decimal128
        .append("saldo", new Decimal128(destinatario.saldo()))
        .append("fecha", Date.from(evento.fecha()));         // la fecha del evento
    String texto = texto(destinatario, evento);
    String encabezado = encabezado(destinatario);
    switch (contacto) {
        case Contacto.Sms sms -> aviso
            .append("canal", "SMS")
            .append("telefono", sms.telefono())
            .append("texto", texto);
        case Contacto.Correo correo -> aviso
            .append("canal", "CORREO")
            .append("correo", correo.correo())
            .append("asunto", encabezado)
            .append("cuerpo", texto);
        case Contacto.Push push -> aviso
            .append("canal", "PUSH")
            .append("token", push.token())
            .append("titulo", encabezado)
            .append("cuerpo", texto);
    }
    return aviso;
}

/** El asunto del correo y el titulo del push: una frase corta segun el tipo. */
static String encabezado(Destinatario destinatario) {
    return switch (destinatario.tipo()) {
        case "TRANSFERENCIA_ENVIADA" -> "Enviaste una transferencia";
        case "TRANSFERENCIA_RECIBIDA" -> "Recibiste una transferencia";
        default -> "Te devolvimos una transferencia";
    };
}
```

documento convierte un evento y un destinatario en un org.bson.Document, que es un mapa de campo a valor. Primero agrega los campos comunes, con monto y saldo como Decimal128 y la fecha del evento (no la de hoy). Después, un switch sobre el Contacto agrega los campos del canal. Como Contacto es una interfaz sellada con solo tres implementaciones (Sms, Correo y Push), el switch tiene que cubrir los tres casos o no compila; por eso no bastaba con el caso de SMS. Para el asunto del correo y el título del push agregué el método encabezado, que da una frase corta según el tipo de aviso, y en cuerpo va el mismo texto de siempre (tipo, monto y saldo resultante).

6.3 Mi código: registrar(evento)

```
public int registrar(EventoTransferencia evento) {
    int nuevos = 0;
    for (Destinatario destinatario : destinatarios(evento)) {
```

```
      Document aviso = documento(evento, destinatario, contactos.de(destinatario.clabe()));
      try {
         mongo.insert(aviso, COLECCION);
         nuevos++;
      } catch (DuplicateKeyException yaEstaba) {
         log.info("[avisos] {} ya estaba: evento repetido, no se duplica", aviso.get("_id"));
      }
   }
   return nuevos;
}
```

Por cada destinatario armo su documento y lo inserto. Si Mongo contesta que el _id ya existe (DuplicateKeyException), ese aviso ya estaba guardado: no es un error, no se cuenta y se sigue con el siguiente. El método regresa cuántos avisos eran nuevos. Inserto uno por uno y no todos juntos porque un insert de varios documentos se detiene en el primer _id repetido: si el proceso se cayó después de guardar el primer aviso, la reentrega tiene que poder guardar el segundo.

6.4 Mi código: crearIndice() y ultimos(clabe)

```
@EventListener(ApplicationReadyEvent.class)
public void crearIndice() {
   String nombre = mongo.indexOps(COLECCION).ensureIndex(
      new Index().on("clabe", Sort.Direction.ASC)
            .on("fecha", Sort.Direction.DESC)
            .named("clabe_fecha"));
   log.info("[avisos] indice {} listo en {}", nombre, COLECCION);
}

public List<Document> ultimos(String clabe) {
   Query consulta = new Query(Criteria.where("clabe").is(clabe))
      .with(Sort.by(Sort.Direction.DESC, "fecha"))
      .limit(ULTIMOS);
   return mongo.find(consulta, Document.class, COLECCION);
}
```

ultimos busca los avisos de una CLABE, ordenados por fecha del más nuevo al más viejo, y se queda con los 10 primeros (ULTIMOS). crearIndice corre una sola vez al arrancar, gracias al @EventListener(ApplicationReadyEvent.class), y crea el índice compuesto clabe_fecha con la misma forma que esa consulta: primero clabe ascendente y luego fecha descendente. Así Mongo encuentra directamente los avisos de esa CLABE y ya los tiene ordenados, sin recorrer toda la colección ni ordenar en memoria. Si el índice ya existe, ensureIndex no hace nada.

6.5 Mi código: el consumidor (una línea)

Archivo:
notificacion/src/main/java/mx/mexibanco/notificacion/mensajeria/ConsumidorDeTransferencias.java

```
// antes: bandeja de Postgres
// nuevos = notificaciones.avisarDeTransferencia(evento).size();
```

```
// ahora: coleccion avisos de MongoDB
nuevos = bitacora.registrar(evento);
```

En recibir cambié la línea que guardaba los avisos en la bandeja de Postgres por bitacora.registrar(evento). Los avisos se mudan, no se copian: guardarlos en Postgres y en Mongo a la vez sería una doble escritura sin transacción común, y si una de las dos fallara quedarían distintas, que es justo el problema que resolvió el Outbox en la S09. El resto del consumidor no cambia: primero se guarda y después se confirma al broker con basicAck.

6.6 Comprobación

Reconstruí solo notificacion con docker compose up --build -d notificacion, y en su log apareció la línea [avisos] indice clabe_fecha listo en avisos.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose logs notificacion | grep avisos
notificacion-1  | 2026-11-06T23:35:32.318Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.BitacoraDeAvisos            :
avisos] indice clabe_fecha listo en avisos
```

Las tres formas. Hice una transferencia de Ana a Beto y otra de Ana a Carla. En mongosh aparecieron cuatro avisos: dos SMS de Ana (uno por cada transferencia que envió), un CORREO de Beto y un PUSH de Carla, cada uno con los campos de su canal y con monto y saldo como Decimal128. Con la consulta find({telefono: {$exists: true}}, ...) solo regresaron los dos avisos de Ana, porque son los únicos documentos que tienen teléfono: el esquema lo aplica quien lee.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
[
  {
    _id: '19833631-7cc9-4827-a150-5233893f2ce2:002180000000000001',
    eventoId: '19833631-7cc9-4827-a150-5233893f2ce2',
    transferenciaId: Long('13'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('200'),
    saldo: Decimal128('580.00'),
    fecha: ISODate('2026-11-06T23:50:23.953Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 200, saldo resultante 580.00'
  },
  {
    _id: '19833631-7cc9-4827-a150-5233893f2ce2:002180000000000002',
    eventoId: '19833631-7cc9-4827-a150-5233893f2ce2',
    transferenciaId: Long('13'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('200'),
    saldo: Decimal128('920.00'),
    fecha: ISODate('2026-11-06T23:50:23.953Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 920.00'
  },
```

```
  {
    _id: 'dd8b8463-3b93-4d52-bc3f-2d1f543b51ec:002180000000000001',
    eventoId: 'dd8b8463-3b93-4d52-bc3f-2d1f543b51ec',
    transferenciaId: Long('14'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50'),
    saldo: Decimal128('530.00'),
    fecha: ISODate('2026-11-06T23:50:29.620Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 530.00'
  },
  {
    _id: 'dd8b8463-3b93-4d52-bc3f-2d1f543b51ec:002180000000000003',
    eventoId: 'dd8b8463-3b93-4d52-bc3f-2d1f543b51ec',
    transferenciaId: Long('14'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50'),
    saldo: Decimal128('350.00'),
    fecha: ISODate('2026-11-06T23:50:29.620Z'),
    canal: 'PUSH',
    token: 'fcm:demo-002180000000000003',
    titulo: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 350.00'
  }
]
```

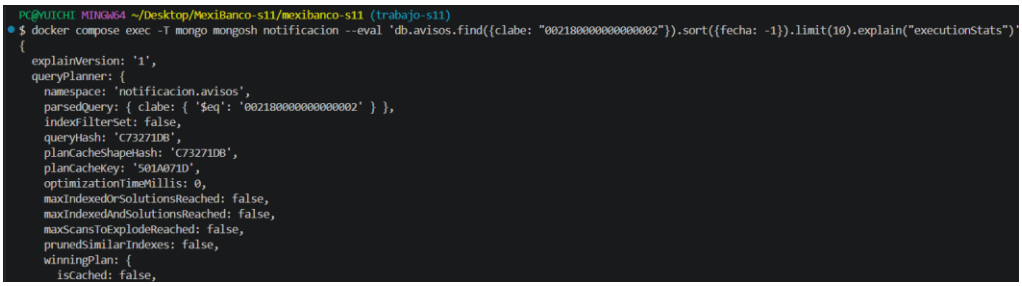
```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: '19833631-7cc9-4827-a150-5233893f2ce2:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'dd8b8463-3b93-4d52-bc3f-2d1f543b51ec:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Los últimos 10. Hice doce transferencias de 1 peso de Ana a Beto y pedí la bitácora de Beto por el gateway con GET /api/notificaciones/002180000000000002. Regresó exactamente 10 avisos, todos de canal CORREO y ordenados del más reciente al más viejo, aunque Beto tenía más de 10 avisos guardados.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/notificaciones/00218000000000002
[{"_id":"15ba6d85-b456-4e73-8a19-9420febf5421:00218000000000002","eventoId":"15ba6d85-b456-4e73-8a19-9420febf5421","transferenciaId":26,"clabe":"002180000000000
02","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":932.00,"fecha":"2026-11-06T23:53:33.024+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibist
e una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 932.00"},{"_id":"9ca84567-eb8a-47ae-93a8-56b6919fc8c4:00218000000000002","eventoId"
:"9ca84567-eb8a-47ae-93a8-56b6919fc8c4","transferenciaId":25,"clabe":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":931.00,"fecha":"2026-1
1-06T23:53:32.927+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultant
e 931.00"},{"_id":"826caa1b-0ef7-47ca-b30a-c25bc072bb03:00218000000000002","eventoId":"826caa1b-0ef7-47ca-b30a-c25bc072bb03","transferenciaId":24,"clabe":"002180
000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":930.00,"fecha":"2026-11-06T23:53:32.829+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto"
:"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 930.00"},{"_id":"b92c6021-e44e-40f3-a45c-4325742222e3:00218000000000002",
"eventoId":"b92c6021-e44e-40f3-a45c-4325742222e3","transferenciaId":23,"clabe":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":929.00,"fech
a":"2026-11-06T23:53:32.733+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo
 resultante 929.00"},{"_id":"b703b1ba-fa8c-4ea9-b890-468eb62b2182:00218000000000002","eventoId":"b703b1ba-fa8c-4ea9-b890-468eb62b2182","transferenciaId":22,"clab
e":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":928.00,"fecha":"2026-11-06T23:53:32.637+00:00","canal":"CORREO","correo":"beto@correo.mx
","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 928.00"},{"_id":"c4b80f41-545c-4f0c-b6a8-06a1e3672298:0021800000
00000002","eventoId":"c4b80f41-545c-4f0c-b6a8-06a1e3672298","transferenciaId":21,"clabe":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":92
7.00,"fecha":"2026-11-06T23:53:32.542+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA po
r 1, saldo resultante 927.00"},{"_id":"91b540df-a0cc-48f3-a812-204887db75a8:00218000000000002","eventoId":"91b540df-a0cc-48f3-a812-204887db75a8","transferenciaId
":20,"clabe":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":926.00,"fecha":"2026-11-06T23:53:32.440+00:00","canal":"CORREO","correo":"beto
@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 926.00"},{"_id":"d09732bc-923a-4b54-83ff-3063c8ebff86:
00218000000000002","eventoId":"d09732bc-923a-4b54-83ff-3063c8ebff86","transferenciaId":19,"clabe":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,
"saldo":925.00,"fecha":"2026-11-06T23:53:32.342+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_R
ECIBIDA por 1, saldo resultante 925.00"},{"_id":"0d3c1a65-152a-475a-8f10-0c3d980b36a1:00218000000000002","eventoId":"0d3c1a65-152a-475a-8f10-0c3d980b36a1","trans
ferenciaId":18,"clabe":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":924.00,"fecha":"2026-11-06T23:53:32.244+00:00","canal":"CORREO","cor
reo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 924.00"},{"_id":"f510825c-a14b-47e2-97da-1d6
bd0f1efb1:00218000000000002","eventoId":"f510825c-a14b-47e2-97da-1d6bd0f1efb1","transferenciaId":17,"clabe":"00218000000000002","tipo":"TRANSFERENCIA_RECIBIDA",
"monto":1,"saldo":923.00,"fecha":"2026-11-06T23:53:32.145+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANS
FERENCIA_RECIBIDA por 1, saldo resultante 923.00"}]
```

El índice trabaja. getIndexes() mostró el índice clabe_fecha además del _id_. Con explain("executionStats"), la consulta de ultimos() usó el plan LIMIT ← FETCH ← IXSCAN y examinó solo 10 documentos (totalDocsExamined: 10). Al obligar a Mongo a no usar el índice con hint({$natural: 1}), el plan cambió a SORT ← COLLSCAN: examinó todos los documentos de la colección y después los ordenó en memoria. Con pocos avisos la diferencia es chica, pero con millones de avisos es la diferencia entre leer 10 documentos y leer la colección completa.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]
```



Continuación de captura en la página siguiente:



```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(10).explain("executionStats")'
    stage: 'LIMIT',
    limitAmount: 10,
    inputStage: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { clabe: 1, fecha: -1 },
        indexName: 'clabe_fecha',
        isMultiKey: false,
        multiKeyPaths: { clabe: [], fecha: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          clabe: [ '["002180000000000002", "002180000000000002"]' ],
          fecha: [ '[MaxKey, MinKey]' ]
        }
      }
    }
  },
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 10,
  executionTimeMillis: 1,
  totalKeysExamined: 10,
  totalDocsExamined: 10,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 10,
    executionTimeMillisEstimate: 0,
    works: 11,
    advanced: 10,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    limitAmount: 10,
    inputStage: {
      stage: 'FETCH',
      nReturned: 10,
      executionTimeMillisEstimate: 0,
      works: 10,
      advanced: 10,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 0,
      docsExamined: 10,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 10,
        executionTimeMillisEstimate: 0,
        works: 10,
        advanced: 10,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 0,
        keyPattern: { clabe: 1, fecha: -1 },
        indexName: 'clabe_fecha',
        isMultiKey: false,
        multiKeyPaths: { clabe: [], fecha: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          clabe: [ '["002180000000000002", "002180000000000002"]' ],
          fecha: [ '[MaxKey, MinKey]' ]
        },
        keysExamined: 10,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  }
},
queryShapeHash: 'CA4FD8110DD5E6683EA9C6BC3C1454D21C0C638D5B978898E268A6EEF92E7759',
command: {
  find: 'avisos',
  filter: { clabe: '002180000000000002' },
  sort: { fecha: -1 },
  limit: 10,
  '$db': 'notificacion'
},
serverInfo: {
  host: '8e02c8a518a7',
  port: 27017,
  version: '8.0.34',
  gitVersion: '51db4515a6597aedcf6188ee4e48a27e65b065ae'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(10).hint({$natural: 1}).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        direction: 'forward'
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 10,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 28,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 10,
      executionTimeMillisEstimate: 0,
      works: 40,
      advanced: 10,
      needTime: 29,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      totalDataSizeSorted: 4530,
      usedDisk: false,
      spills: 0,
      spilledDataStorageSize: 0,
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        nReturned: 13,
        executionTimeMillisEstimate: 0,
        works: 29,
        advanced: 13,
        needTime: 15,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        direction: 'forward',
        docsExamined: 28
      }
    }
  },
  queryShapeHash: 'CA4FD8110DD5E6683FA9C6BC3C14E4D21C0C63BD58978B90E268A6EEF92E7759',
  command: {
    find: 'avisos',
    filter: { clabe: '002180000000000002' },
    sort: { fecha: -1 },
    hint: { '$natural': 1 },
    limit: 10,
    '$db': 'notificacion'
  },
  serverInfo: {
    host: '8e82c8a518a7',
    port: 27017,
    version: '8.0.34',
    gitVersion: '51db4515a6597aedcf6188ce4e48a27e65b065ae'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```

7. Parte 3: rompe algo, el evento duplicado

La mensajería de la S09 entrega «al menos una vez», así que un mismo evento puede llegar dos veces. Lo provoqué publicando dos veces el mismo evento TransferenciaRealizada (el mismo eventoId, con las CLABEs inventadas …091 y …092 para no mezclarlo con mis cuentas) directamente en el exchange transferencias, usando la API HTTP del panel de RabbitMQ.

Predicción: dos eventos con dos avisos cada uno podrían dejar cuatro documentos, pero como el _id es eventoId:clabe, el segundo evento produce exactamente los mismos dos _id y Mongo los rechaza. Deben quedar 2.

Resultado: las dos publicaciones respondieron {"routed":true}, countDocuments para ese eventoId dio 2 y en el log de notificación aparecieron dos líneas «ya estaba: evento repetido, no se duplica», una por cada CLABE. La idempotencia ya no necesita una tabla aparte de eventos procesados: la da la llave primaria del documento.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish -H 'Content-Type: application/json' -d '{"properties":{"deli
very_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-
8444-555555555555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"
monto\":25,\"saldoOrigen\":475,\"saldoDestino\":525,\"fecha\":\"2026-10-31T13:00:00Z\"}"}'
{"routed":true}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-                Follow link (ctrl + click)                abajo-s11)
$ curl -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish -H 'Content-Type: application/json' -d '{"properties":{"deli
very_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-
8444-555555555555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"
monto\":25,\"saldoOrigen\":475,\"saldoDestino\":525,\"fecha\":\"2026-10-31T13:00:00Z\"}"}'
{"routed":true}
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555"})'
2

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose logs notificacion | grep "ya estaba"
notificacion-1  | 2026-11-07T00:01:52.816Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           : [avisos] 11111111-2222-4333-844
4-555555555555:002180000000000091 ya estaba: evento repetido, no se duplica
notificacion-1  | 2026-11-07T00:01:52.818Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           : [avisos] 11111111-2222-4333-844
4-555555555555:002180000000000092 ya estaba: evento repetido, no se duplica
```

8. Parte 4: rompe algo, el dato viejo

La caché puede mentir. Lo provoqué cambiando el saldo de Ana directamente en Postgres con psql, como lo haría un script mal hecho. Así el cambio no pasa por @CacheEvict y nadie borra la copia de Redis.

1. Consulté a Ana para que su saldo quedara en la caché y anoté cuánto tenía.
2. Con un UPDATE directo en la tabla cuenta puse su saldo en 5.
3. Cada 5 segundos revisé el TTL de la copia y el saldo que veía el cliente.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":518.00,"leidaEn":"2026-11-07T00:03:34.646988632Z"}
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 5 where clabe = '002180000000000001'"
UPDATE 1
```

Resultado: mientras la copia seguía viva, el GET siguió mostrando el saldo de antes (cientos de pesos) con el mismo leidaEn, aunque en Postgres ya había 5. Solo cuando el TTL llegó a -2 (la clave ya no existe) la siguiente consulta fue a Postgres y mostró 5.00 con un leidaEn nuevo. El cliente vio un saldo que ya no existía durante todo lo que le quedaba de vida a la copia, cerca de 30 segundos.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ for i in $(seq 1 8); do docker compose exec -T redis redis-cli TTL cuentas::002180000000000001; curl -s http://localhost:8080/api/cuentas/002180000000000001; echo;
 sleep 5; done
-2
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:14.297014876Z"}
25
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:14.297014876Z"}
19
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:14.297014876Z"}
14
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:14.297014876Z"}
9
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:14.297014876Z"}
3
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:14.297014876Z"}
-2
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:46.657841431Z"}
25
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T00:04:46.657841431Z"}
```

Mientras el saldo viejo seguía en la caché, desde otra terminal intenté transferir 100 de Ana a Beto. **Predicción:** aunque la app le muestra a Ana cientos de pesos, la transferencia no debe pasar. **Resultado:** respondió 422 por saldo insuficiente, porque la autorización del cargo (CuentaService.consultar) lee Postgres, donde Ana solo tenía 5. Además, como el cargo falló, @CacheEvict no se ejecutó y la copia vieja siguió ahí hasta caducar. La caché mostró un dato falso, pero no decidió nada: el dinero estuvo a salvo.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180000000000001","claveDestino":"00218000000
0000002","monto":100}'
HTTP/1.1 422 Unprocessable Entity
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sat, 07 Nov 2026 00:04:27 GMT

{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180000000000001","instance":"/transferencias","servicio
":"cuenta"}
```

Al terminar regresé el saldo de Ana a su valor original con el mismo UPDATE. Hay que tener en cuenta que este UPDATE rompe a propósito el cuadre entre el saldo y los movimientos; en un sistema real el saldo nunca se debe cambiar por fuera de la aplicación.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 518.00 where clabe = '002180000000000001'"
UPDATE 1

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T db psql -U mexibanco -d cuenta -c "select clabe, saldo from cuenta where clabe = '002180000000000001'"
       clabe        | saldo
--------------------+--------
 002180000000000001 | 518.00
(1 row)
```

9. Reto: Redis se cae

La caché era solo una copia para mostrar, así que apagar Redis no debería tumbar nada. Para comprobarlo apagué solo Redis con docker compose stop redis, dejando todo lo demás arriba.

9.1 Lo que pasaba en mi versión de la Parte 1

Con Redis apagado, el GET del saldo respondió 500. @Cacheable intentó leer Redis, la operación falló después de 500 ms (el spring.data.redis.timeout de application.yml) y, como el manejador de errores de caché por omisión vuelve a lanzar la excepción, la consulta entera falló aunque Postgres estaba bien. La transferencia tampoco se completó: la saga primero revisa que existan las dos cuentas con ese mismo GET y, al recibir un error de cuenta, la rechazó antes de mover dinero. Una copia que solo sirve para mostrar terminó tumbando la consulta y las transferencias.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose stop redis
[+] stop 1/1
 ✓ Container mexi-banco-v06-redis-1 Stopped                                    0.3s

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s -i http://localhost:8080/api/cuentas/00218000000000001
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
X-Instancia: 5c638a24f91c
Content-Type: application/json
Date: Sat, 07 Nov 2026 00:24:20 GMT

{"timestamp":"2026-11-07T00:24:20.667+00:00","status":500,"error":"Internal Server Error","path":"/cuentas/002180000000000001"}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000
0000002","monto":10}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sat, 07 Nov 2026 00:24:41 GMT

{"type":"about:blank","title":"Internal Server Error","status":500,"detail":"El servicio cuenta respondio 500","instance":"/transferencias","servicio":"cuenta"}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose start redis
[+] start 1/1
 ✓ Container mexi-banco-v06-redis-1 Started                                    0.2s
```

9.2 Dónde atrapar el error

Lo primero que pensé fue un CacheErrorHandler, que Spring llama cuando falla una operación de caché. Ese manejador sirve para el GET y el SET, que ocurren dentro de la llamada al método. Pero con transactionAware() el DEL de @CacheEvict no ocurre ahí: solo se registra, y se ejecuta después, en el afterCommit de la transacción. Si Redis falla en ese momento, la excepción sale del commit, por fuera del CacheErrorHandler, y cuenta contesta 500 a un cargo que ya quedó confirmado en Postgres.

Eso sería grave para la saga. Si pasa en el cargo, la saga lo toma como un cargo rechazado: marca la transferencia como FALLIDA y no devuelve nada, así que a Ana se le descuenta el dinero sin que llegue a Beto. Si pasa en el abono, la saga compensa un abono que sí ocurrió: Ana recibe la devolución y Beto se queda con el dinero, es decir, el dinero se duplica.

Por eso atrapé el error más abajo, en la caché misma. Hice CacheTolerante, que envuelve la RedisCache y atrapa los errores de Redis en cada operación. El orden de los envoltorios es lo importante:

```
TransactionAwareCacheDecorator -> CacheTolerante -> RedisCache
```

El decorador transaccional queda por fuera, así que el DEL sigue esperando al commit como en la Parte 1. Cuando por fin se ejecuta, pasa por CacheTolerante, que atrapa el error y solo lo anota en el log, y el cargo o el abono responde bien. Un GET que falla cuenta como miss (se lee Postgres) y un SET que falla simplemente no guarda la copia.

9.3 Mi código: CacheTolerante.java (archivo nuevo)

Archivo: cuenta/src/main/java/mx/mexibanco/cuenta/CacheTolerante.java

```
class CacheTolerante implements Cache {

  private static final Logger log = LoggerFactory.getLogger(CacheTolerante.class);
  private final Cache redis;

  CacheTolerante(Cache redis) { this.redis = redis; }

  @Override public String getName() { return redis.getName(); }
  @Override public Object getNativeCache() { return redis.getNativeCache(); }

  @Override
  public ValueWrapper get(Object key) {
    try {
      return redis.get(key);
    } catch (RuntimeException falla) {
      avisar("GET", key, falla);
      return null;                  // como un miss: se lee Postgres
    }
  }

  @Override
  public void put(Object key, Object value) {
    try {
      redis.put(key, value);
    } catch (RuntimeException falla) {
      avisar("SET", key, falla);       // no se guarda la copia y ya
    }
  }

  @Override
  public void evict(Object key) {
    try {
      redis.evict(key);
    } catch (RuntimeException falla) {
      avisar("DEL", key, falla);       // el cargo o abono ya hizo commit
    }
  }

  // get(key, type) y clear() igual: try/catch y avisar(...)

  private void avisar(String operacion, Object key, RuntimeException falla) {
    log.warn("[cache] fallo {} {}::{} ({}: {}); sigo sin cache",
```

```
            operacion, getName(), key, falla.getClass().getSimpleName(), falla.getMessage());
    }
}
```

9.4 Mi código: cambios en ConfiguracionCache.java

```
@EnableCaching
@Configuration
public class ConfiguracionCache implements CachingConfigurer {
    ...
    // en el customizer ya NO va .transactionAware(): ahora lo pongo yo en cacheResolver()
    return builder -> builder.withCacheConfiguration(CUENTAS, cuentas);
    ...
    @Override
    public CacheResolver cacheResolver() {
        return contexto -> contexto.getOperation().getCacheNames().stream()
            .map(this::envolver)
            .toList();
    }

    private Cache envolver(String nombre) {
        // "cacheManager" es el que arma Spring Boot: Redis, o NoOp con CACHE_TIPO=none
        CacheManager manager = beans.getBean("cacheManager", CacheManager.class);
        return new TransactionAwareCacheDecorator(new CacheTolerante(manager.getCache(nombre)));
    }
}
```

ConfiguracionCache ahora implementa CachingConfigurer y da su propio CacheResolver, que es lo que Spring usa para saber con qué caché trabajan @Cacheable y @CacheEvict. El resolver toma la caché que arma Spring Boot y la envuelve en el orden de arriba. Quité .transactionAware() del builder porque ahora el decorador transaccional lo pongo yo, por fuera de CacheTolerante; si lo dejara, quedaría un segundo decorador transaccional por dentro. Ese decorador se registraría durante el afterCommit, cuando ya no se ejecuta, y el DEL se perdería. CACHE_TIPO=none sigue funcionando, porque en ese caso el resolver envuelve la caché NoOp y no pasa nada.

¿Y si el DEL falla pero Redis en realidad sigue vivo con la copia vieja? Entonces el saldo viejo se vería a lo mucho 30 segundos, por el TTL: el mismo riesgo acotado de la Parte 4, y la caché sigue sin decidir nada. Si Redis se cayó de verdad, como guarda solo en memoria, al volver arranca vacío y no queda ninguna copia vieja.

9.5 Comprobación

Con Redis encendido reconstruí cuenta y anoté los saldos de Ana y Beto. Después apagué Redis y repetí las pruebas.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose ps cuenta
NAME                      IMAGE                     COMMAND              SERVICE   CREATED          STATUS                    PORTS
mexi-banco-v06-cuenta-1   mexi-banco-v06-cuenta     "java -jar app.jar"  cuenta    15 seconds ago   Up 10 seconds (healthy)   8080/tcp
mexi-banco-v06-cuenta-2   mexi-banco-v06-cuenta     "java -jar app.jar"  cuenta    15 seconds ago   Up 10 seconds (healthy)   8080/tcp
mexi-banco-v06-cuenta-3   mexi-banco-v06-cuenta     "java -jar app.jar"  cuenta    15 seconds ago   Up 10 seconds (healthy)   8080/tcp
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001; echo; curl -s http://localhost:8080/api/cuentas/002180000000000002; echo
{"clabe":"002180000000000001","titular":"Ana","saldo":518.00,"leidaEn":"2026-11-07T00:28:56.013022395Z"}
{"clabe":"002180000000000002","titular":"Beto","saldo":932.00,"leidaEn":"2026-11-07T00:28:56.394982484Z"}
```

Resultado: con Redis apagado, el GET respondió 200 con el saldo leído de Postgres. Fue un poco más lento, porque cada operación de caché espera hasta 500 ms antes de darse por fallida. La transferencia de 10 de Ana a Beto respondió 201 con estado COMPLETADA, y los saldos cuadraron: Ana bajó 10 y Beto subió 10. En el log de cuenta aparecieron las líneas [cache] fallo GET, [cache] fallo SET y [cache] fallo DEL, con la clave y el tipo de error, así que queda claro qué operación de caché falló.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s -i http://localhost:8080/api/cuentas/002180000000000001
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 29a6fc6ca059
Content-Type: application/json
Date: Sat, 07 Nov 2026 00:29:32 GMT

{"clabe":"002180000000000001","titular":"Ana","saldo":518.00,"leidaEn":"2026-11-07T00:29:32.336109408Z"}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180000000000001","claveDestino":"00218000000
0000002","monto":10}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 07 Nov 2026 00:29:48 GMT

{"id":28,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10,"fecha":"2026-11-07T00:29:48.662205402Z","estado":"COMPLETADA","motivo":nu
ll,"saldoOrigenTrasCargo":508.00,"intentosDeCompensacion":0,"actualizada":"2026-11-07T00:29:49.886568443Z","eventoId":"a89cff62-7d6a-46bc-9bc3-cdac7e7a5b61"}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001; echo; curl -s http://localhost:8080/api/cuentas/002180000000000002; echo
{"clabe":"002180000000000001","titular":"Ana","saldo":508.00,"leidaEn":"2026-11-07T00:30:08.980708660Z"}
{"clabe":"002180000000000002","titular":"Beto","saldo":942.00,"leidaEn":"2026-11-07T00:30:09.530863228Z"}
```

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose logs --since 3m cuenta | grep -F "[cache]"
cuenta-3  | 2026-11-07T00:29:32.332Z  WARN 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo GET cuentas::002180000000000001
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-3  | 2026-11-07T00:29:32.836Z  WARN 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo SET cuentas::002180000000000001
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-1  | 2026-11-07T00:29:49.810Z  WARN 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo GET cuentas::002180000000000002
(RedisConnectionFailureException: Unable to connect to Redis); sigo sin cache
cuenta-1  | 2026-11-07T00:29:49.812Z  WARN 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo DEL cuentas::002180000000000002
(RedisConnectionFailureException: Unable to connect to Redis); sigo sin cache
cuenta-1  | 2026-11-07T00:29:49.830Z  WARN 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo SET cuentas::002180000000000002
(RedisConnectionFailureException: Unable to connect to Redis); sigo sin cache
cuenta-1  | 2026-11-07T00:30:08.978Z  WARN 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo GET cuentas::002180000000000001
(RedisConnectionFailureException: Unable to connect to Redis); sigo sin cache
cuenta-3  | 2026-11-07T00:29:48.157Z  WARN 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo GET cuentas::002180000000000002
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-3  | 2026-11-07T00:29:48.660Z  WARN 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo SET cuentas::002180000000000002
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-1  | 2026-11-07T00:30:08.981Z  WARN 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo SET cuentas::002180000000000001
(RedisConnectionFailureException: Unable to connect to Redis); sigo sin cache
cuenta-2  | 2026-11-07T00:29:45.139Z  WARN 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo GET cuentas::002180000000000001
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-2  | 2026-11-07T00:29:45.643Z  WARN 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo SET cuentas::002180000000000001
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-2  | 2026-11-07T00:29:49.241Z  WARN 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo DEL cuentas::002180000000000001
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-2  | 2026-11-07T00:30:09.528Z  WARN 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo GET cuentas::002180000000000002
(QueryTimeoutException: Redis command timed out); sigo sin cache
cuenta-2  | 2026-11-07T00:30:10.031Z  WARN 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CacheTolerante      : [cache] fallo SET cuentas::002180000000000002
(QueryTimeoutException: Redis command timed out); sigo sin cache
```

Al encender Redis otra vez con docker compose start redis, después de unos segundos (lo que tarda el cliente Lettuce en reconectarse) la caché volvió a funcionar: la primera consulta llenó la copia, EXISTS dio 1 y ya no aparecieron errores de caché en el log.

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ for i in 1 2 3; do curl -s http://localhost:8080/api/cuentas/002180000000000001; echo; done
{"clabe":"002180000000000001","titular":"Ana","saldo":508.00,"leidaEn":"2026-11-07T00:31:54.912305381Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":508.00,"leidaEn":"2026-11-07T00:31:54.912305381Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":508.00,"leidaEn":"2026-11-07T00:31:54.912305381Z"}

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001
1

PC@YUICHI MINGW64 ~/Desktop/MexiBanco-s11/mexibanco-s11 (trabajo-s11)
$ docker compose logs --since 20s cuenta | grep -cF "[cache]"
0
```

10. Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?

Por la forma del dato y por lo que tiene que garantizar. Cada aviso tiene campos distintos según su canal (teléfono y texto; correo, asunto y cuerpo; token, título y cuerpo), se escribe una sola vez, nunca se actualiza y se lee entero, por CLABE y fecha. No hay joins ni reglas entre avisos. Eso es justo lo que hace bien un documento: cada uno con su propia forma, sin columnas vacías ni ALTER TABLE, con un índice que sigue a la consulta y con el _id como llave natural para no duplicar.

El saldo es lo contrario: es dinero, cambia con cada cargo y abono, tiene que cuadrar con los movimientos y necesita transacciones, @Version contra escrituras concurrentes e idempotencia guardada en la misma transacción que el cambio de saldo (como en la S08). Todo eso ya lo garantiza PostgreSQL, y moverlo a otro motor solo agregaría riesgo. Nada que sea dinero cambia de motor.

2. ¿Qué hit ratio medí en A y en B, y por qué cambia? ¿Qué compré con la caché, velocidad o capacidad?

En el escenario A medí alrededor del 99 % (todas hits menos la primera consulta) y en el B alrededor del 84 %. Cambia porque cada transferencia borra las copias de Ana y de Beto, y la siguiente lectura de cada uno es un miss; mientras más escrituras hay por cada lectura, menos aciertos tiene la caché.

Con mis números, la latencia mejoró poco, apenas unos milisegundos, porque el costo de cada petición está sobre todo en el gateway y en la red. En cambio, las lecturas a Postgres bajaron de 100 a una por cada 100 consultas. Lo que compré fue **capacidad**: Postgres queda libre para lo que solo él puede hacer, que son los cargos y los abonos, y el sistema aguanta muchas más consultas de saldo con la misma base.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

En la Parte 4 la caché mostró durante casi 30 segundos un saldo de cientos de pesos cuando en Postgres Ana solo tenía 5. Si el cargo leyera de la caché, la transferencia de 100 se habría autorizado con ese saldo viejo y Ana habría quedado en negativo (5 − 100 = −95): el banco habría dejado salir dinero que no existía. Y no hace falta un UPDATE manual para que pase; cualquier copia puede estar atrasada respecto a la verdad. Por eso la regla es que la caché muestra, pero nunca decide: la lectura que autoriza siempre va a Postgres.

4. ¿Por qué DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit?

DEL y no SET. Borrar es más simple y más seguro. Con SET la aplicación tendría que escribir el valor nuevo en dos sistemas sin transacción común: si dos operaciones concurrentes hacen su SET en distinto orden que sus commits, la caché se queda con un saldo que no es el último; si el SET falla, se queda el viejo; y si la transacción hace rollback después del SET, la caché tendría un saldo que nunca existió. Con DEL nunca se escribe en la caché un valor que no salga de leer Postgres: la siguiente consulta lee la verdad y vuelve a llenar la copia. Además, si un dato se escribe mucho y se lee poco, no se gasta trabajo en llenar copias que nadie va a leer.

Después del commit y no antes. Si el DEL ocurriera antes del commit, en la ventana entre el borrado y el commit otra consulta podría hacer miss, leer de Postgres el saldo viejo (porque la transacción todavía no se confirma) y volver a guardarlo en la caché, donde se quedaría hasta 30 segundos. Por eso transactionAware() hace que el DEL espere al commit: cuando se borra la copia, Postgres ya tiene el saldo nuevo y cualquier miss posterior lee el valor correcto.

11. Conclusiones

- Persistencia políglota no significa pasar todo a NoSQL, sino escoger el motor según el dato: PostgreSQL para el dinero (la verdad), Redis para una copia del saldo que solo se muestra y MongoDB para avisos de forma variable que se escriben una vez.
- La caché compró sobre todo capacidad: las lecturas de saldo a Postgres casi desaparecieron, y las tres réplicas de cuenta comparten la misma copia porque Redis está fuera de ellas.
- Una copia siempre puede estar atrasada. El TTL limita cuánto puede mentir (30 segundos) y el borrado después del commit evita que guarde datos viejos en el camino normal, pero lo que decide, como el cargo, siempre debe leer la verdad.
- Con un _id natural (eventoId:clabe), MongoDB vuelve idempotente al consumidor sin una tabla adicional, lo cual es necesario porque RabbitMQ entrega al menos una vez.
- Los avisos se mudaron de Postgres a Mongo, no se copiaron: escribir lo mismo en dos bases sin una transacción común termina, tarde o temprano, en datos distintos.

- Si la caché es solo una copia, su caída no debe tumbar nada. Con el reto, cuenta sigue funcionando sin Redis, y como el error se atrapa debajo del decorador transaccional, nunca se contesta 500 a un cargo o abono que ya hizo commit.